面向Ascend 950的8bit量化矩阵乘性能建模与优化方法论

分享嘉宾：杨阳

2026.4.18　中国·北京

01 8bit量化基础知识

02 矩阵乘性能建模方法

03 矩阵乘优化技术

04 加入CANN社区

content

目录

8bit量化基础知识

01

1.1 NPU内存层次结构与计算单元

内存层次结构(Ascend 950)

Global Memory

L2 Cache

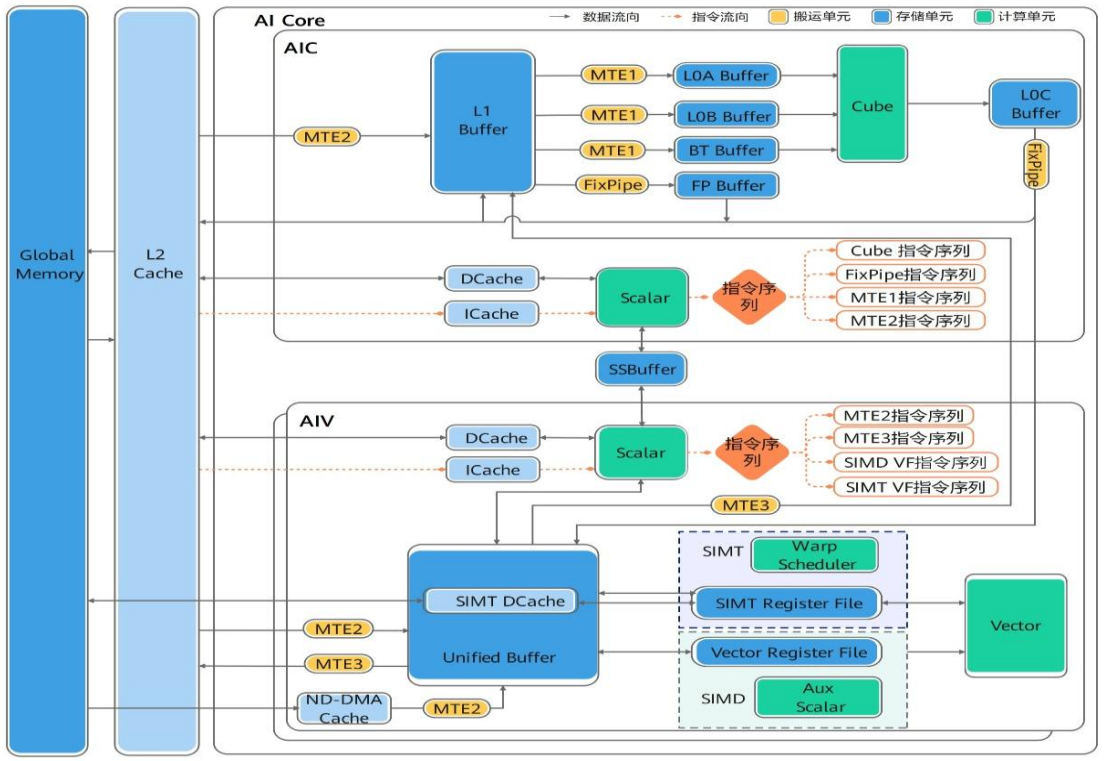
L1 Buffer

L0 Buffer

Unified Buffer

Ascend 950 CUBE峰值算力(以32核为例)

- BF16 算力: 432 TFLOPS
- INT8 算力 : 865 TFLOPS
- FP8 算力 : 865 TFLOPS
- MXFP8 算力 : 865 TFLOPS
- HiF8 算力 : 865 TFLOPS

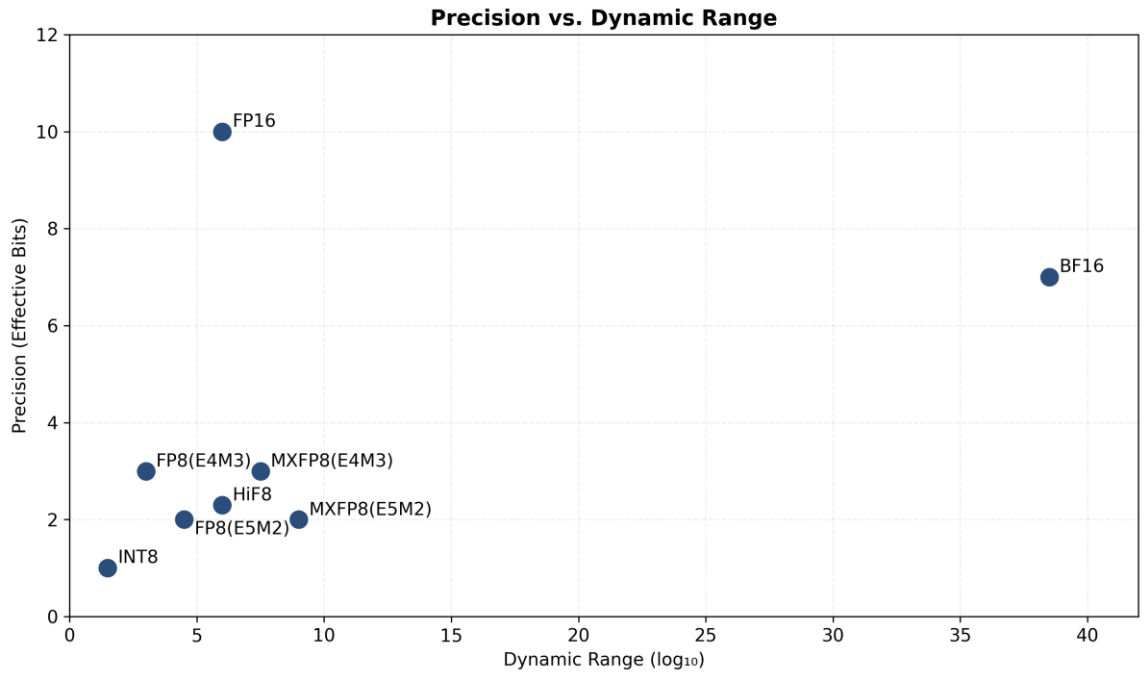


关键: 矩阵乘法优化核心在于 最大化平衡计算和访存 ，优化的手段一般按照访存效率优化、计算效率优化、指令并行度优化来展开。

1.2 8bit量化格式对比

格式特性对比

格式	指数位	尾数位	动态范围
INT8	N/A	8-bit	[-128, 127]
FP8 E4M3	4	3	$[2^{-9},\ 1.75 \times 2^{8}]$
FP8 E5M2	5	2	$[2^{-16},\ 1.5 \times 2^{15}]$
MXFP8(E4M3)	4+共享	3	块级缩放
MXFP8(E5M2)	5+共享	2	块级缩放
HiF8	动态	动态	$[2^{-22}, 2^{15}]$



精度-性能权衡

① **Ascend 950支持:** 950原生新增支持FP8/MXFP8/HiF8多种8bit算力格式。

② **精度:** 相比INT8，新增的FP8/MXFP8/HiF8格式动态范围更广，有效精度更高，在保持8bit位宽的同时，兼容支持训练和推理场景。

③ **性能:** 训练场景下，HiF8量化性能优势明显，使用pertensor量化就能支持精度达标；推理场景下量化方案选择丰富，HiF8 与 MXFP8 凭借纯 Cube 核硬件支持，可实现高效 8bit 量化推理。

Microscaling (MX) 格式: 通过 块级共享指数（Block-wise Shared Exponent）扩展动态范围，MXFP8块大小为32，在每个块内使用共享的E8M0缩放因子，结合E4M3/E5M2尾数表示，实现比标准FP8更优的精度-范围平衡。

HiF8格式: HiF8利用即时可译的 变长前缀码 编码点位域Dot，实现符合AI数据分布特征的锥形精度格式。综合阶码接近FP16的40个阶码值表达。HiF8兼顾精度和动态范围，为神经网络的训练和推理，提供了能力更全面的8-bit单数据格式表达。

1.3 量化方案对比

量化粒度介绍

Per-Tensor
整个张量一个scale，最粗粒度，计算开销最小。

Per-Channel
每通道独立scale，权重常用策略，精度优于Per-Tensor，适合权重矩阵量化。

Per-Token
每token独立scale, 激活值动态量化，需在线计算scale，增加计算开销。

Per-Group
在reduce轴上对数据分组，每组使用独立的量化参数，细粒度量化方案，是MX量化的核心基础。

MX
基于Per-Group（32）量化的完整低比特计算标准，昇腾芯片原生支持，精度和性能较好平衡

量化方案对比

通用规律:
量化方案的本质差异，是量化粒度配套量化格式的差异，粒度越细，对数据分布的适配性越好、精度损失越小，但 scale 的存储开销、硬件实现复杂度越高；反之粒度越粗，实现越简单、开销越小，但精度天花板越低。

昇腾量化方案推荐

HiF8-Pertensor与MXFP8是面向大模型训推的推荐方案，深度适配昇腾硬件原生计算能力，在精度保持、性能竞争力、训推一致性上有显著优势。在网络精度评估达标前提下，推荐使用。

HiF8-Pertensor:
无需细粒度量化即可稳定收敛，昇腾芯片原生支持，纯Cube核计算，使用极简量化，计算开销最低。

MXFP8:
标准格式，方便跨平台迁移，昇腾芯片原生支持，纯Cube核计算，Scale乘法融合到MMAD指令，零额外耗时。

矩阵乘性能建模方法

02

2.1 性能建模方法

为什么需要建模

传统调优方式往往依赖经验或大量试错。性能建模的核心价值在于将调优过程**量化、可预测、可迁移**:

方式	特点	问题
经验调优	根据直觉选择优化策略	经验难以复用，新场景需重新摸索
试错调优	尝试多种配置选最优	时间成本高，可能错过最优解
性能建模	量化分析各流水耗时	针对性优化，方法可迁移

理论耗时公式(以MXFP8为例)

MMAD: $T_{\text{cube}} = \frac{M \times K \times N}{16 \times 32 \times 16 \times \text{核数} \times \text{频率}}$

GM2L1: $T_{\text{mte2}} \approx \frac{(M \times \frac{N}{\text{baseN}} + N \times \frac{M}{\text{baseM}}) \times K \times (sizeof(dtype) + \frac{1}{32})}{BandWidth_{mte2}}$

L12L0: $T_{\text{mte1}} \approx \frac{baseM \times baseK \times (sizeof(dtype) + \frac{1}{32})}{BandWidth_{L0A}} + \frac{baseN \times baseK \times (sizeof(dtype) + \frac{1}{32})}{BandWidth_{L0B}}$

L02GM: $T_{fixp} = \frac{M \times N \times sizeof(dtype)}{BandWidth_{\text{fixp}}}$

性能瓶颈分析

Memory Bound

当$\max(T_{mte2}, T_{mte1}, T_{fixp}) > T_{cube}$，搬运受限，优化方向为减少搬运量、提高带宽利用率；常见优化手段有基本块调整、L1全载、增加L2命中率、WeightNZ、解决同地址冲突、双页表等。

Cube Bound

当 $T_{cube} \geq \max(T_{mte2}, T_{mte1}, T_{fixp})$，计算已达极限，优化方向关注负载均衡。常见优化手段有多核切K、尾轮负载均衡。

Bound类型不清晰

当流水图上识别到流水阻塞、Bound类型不清晰。可以考虑从指令并行度入手优化，常见优化手段有Double Buffer、Unitflag等

关键: 通过定量分析，精准定位瓶颈因素，避免“盲人摸象”式的调优。通过性能瓶颈分析，解决访存效率或计算效率问题，从而达成Memory Bound或Cube Bound，充分发挥硬件资源能力。

矩阵乘优化技术

03

3.1 搬运效率优化：SWAT(自适应滑动窗口模板)

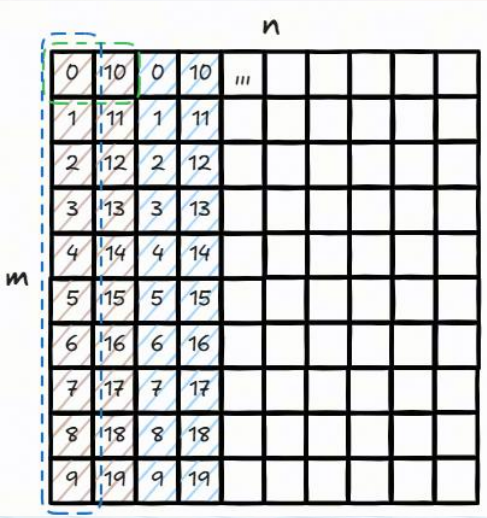
原理介绍

- SWAT(Slide Window Adaptive Template)通过提升多核单次访问的L2命中率来提高MTE2搬运效率，从而实现首轮搬运即可做到MMAD指令不断流。
- 核心逻辑是使每一轮多核计算的输出排布尽量方正，具体方法是在M轴上设定固定窗口，根据M轴方向尾块大小灵活调整，再沿N方向进行"Z"型滑动，从而最大程度提高L2命中率。

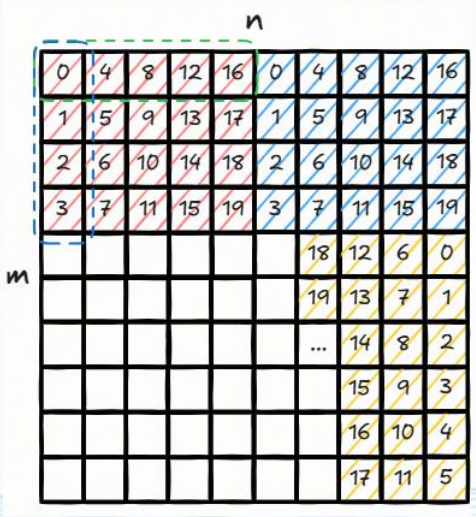
适用场景

1. 大规模矩阵乘法场景
2. 多核并行计算场景
3. 首轮Cube流水存在断流的场景

列优先搬运



SWAT



列优先搬运

- 首轮20核数据，左矩阵10份数据首次访问，10份数据重复访问，右矩阵2份数据首次访问，18份数据重复访问，L2命中率 = (10 + 18) / 40 = 70%
- 次轮计算，左矩阵全部命中L2，仅有右矩阵的2份数据首次访问，L2命中率 = (20 + 18) / 40 = 95%

SWAT

- 左矩阵4份数据首次访问，16份数据重复访问，右矩阵5份数据首次访问，15份数据重复访问，L2命中率 = (15 + 16) / 40 = 77.5%
- 次轮计算，左矩阵全部命中L2，仅有右矩阵的5份数据首次访问，L2命中率 = (20 + 15) / 40 = 87.5%
- 窗口会随着剩余的行数自动调整，使用“Z”型走位，提高换行时的L2命中率。

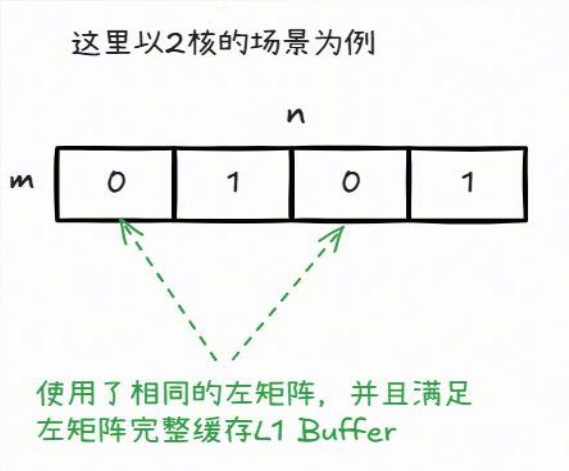
3.2 搬运效率优化：全载优化

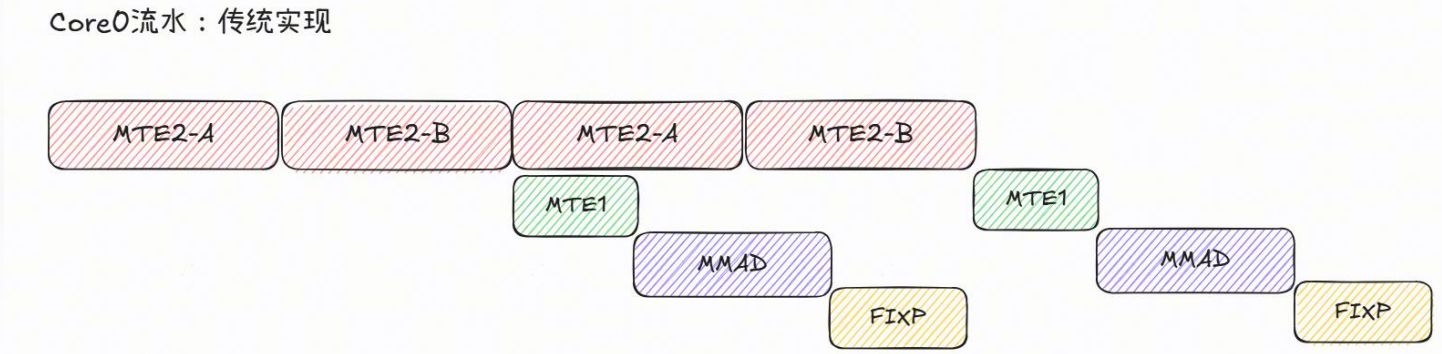
原理介绍

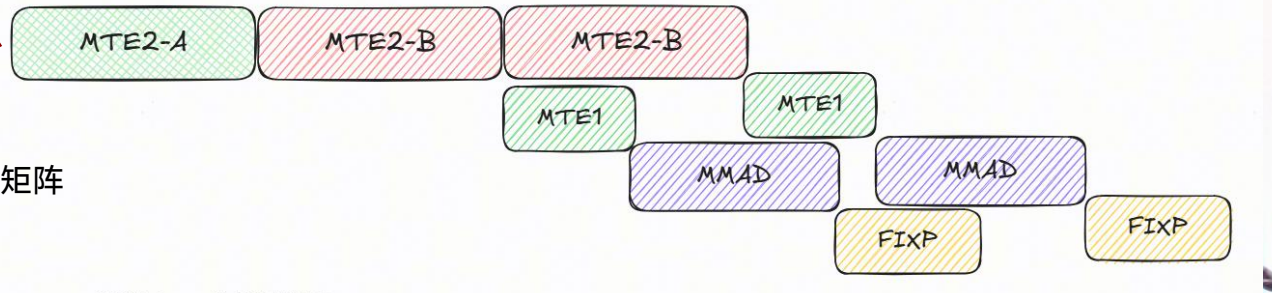
- 在MTE2 Bound场景中，可通过减少MTE2部分的整体搬运量来提高算子性能。
- 对于输入可完整缓存在L1中的场景，可使用全载模板，在不同轮询中使输入始终驻留在L1中，从而减少整体搬运耗时。

适用场景

1. MTE2 Bound为主要瓶颈的场景。
2. Decode等小批量计算场景。







左矩阵驻留L1，和不同的右矩阵完成矩阵乘，减少了MTE2-A的搬运次数

Core0流水：全载模板

3.3 计算效率优化：尾核负载均衡

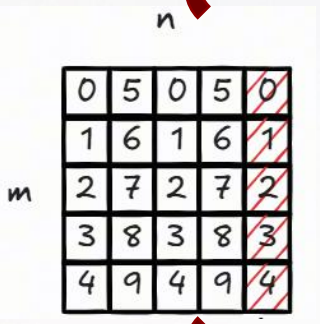
原理介绍

- 当前Matmul算子普遍使用基本块策略进行多核分配。但在处理不同规格输入时，划分的基本块无法均匀分配到所有核上，导致分核不均，尤其是最后一轮计算存在算力浪费，使整体算力利用率无法达到最优。
- 将最后一轮未完全分配的基本块进行二次切分，使其尽量均匀分配到多核中，充分发挥完整算力。

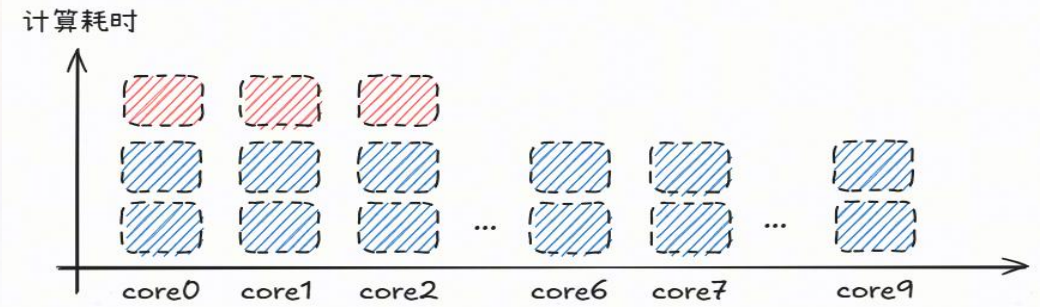
适用场景

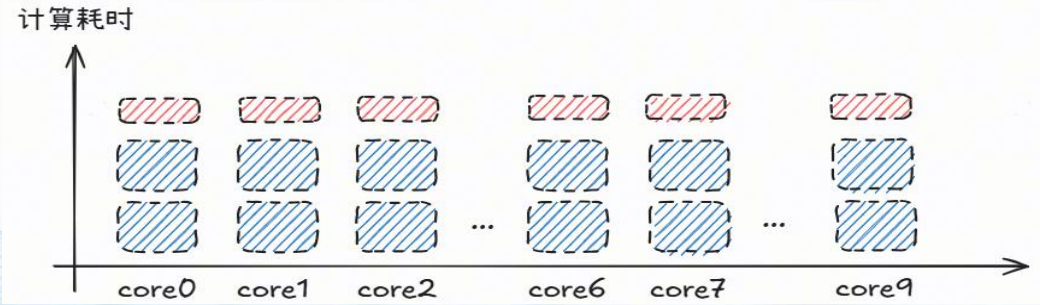
1. 多核并行计算场景。
2. 输入形状不规则，无法均匀分配的场景。
3. 最后一轮计算存在算力浪费的场景。

无负载均衡



尾轮负载均衡





无负载均衡

- 假设物理核数为10核，前两轮计算可分配到10个核上。
- 最后一轮无法均匀分配，仅使用5个核，存在算力浪费。

尾轮负载均衡

- 最后一轮在N维度上进行二次切分，从而均匀分配给所有核。
- 尾轮计算被均匀分配给所有核，从而降低端到端耗时。

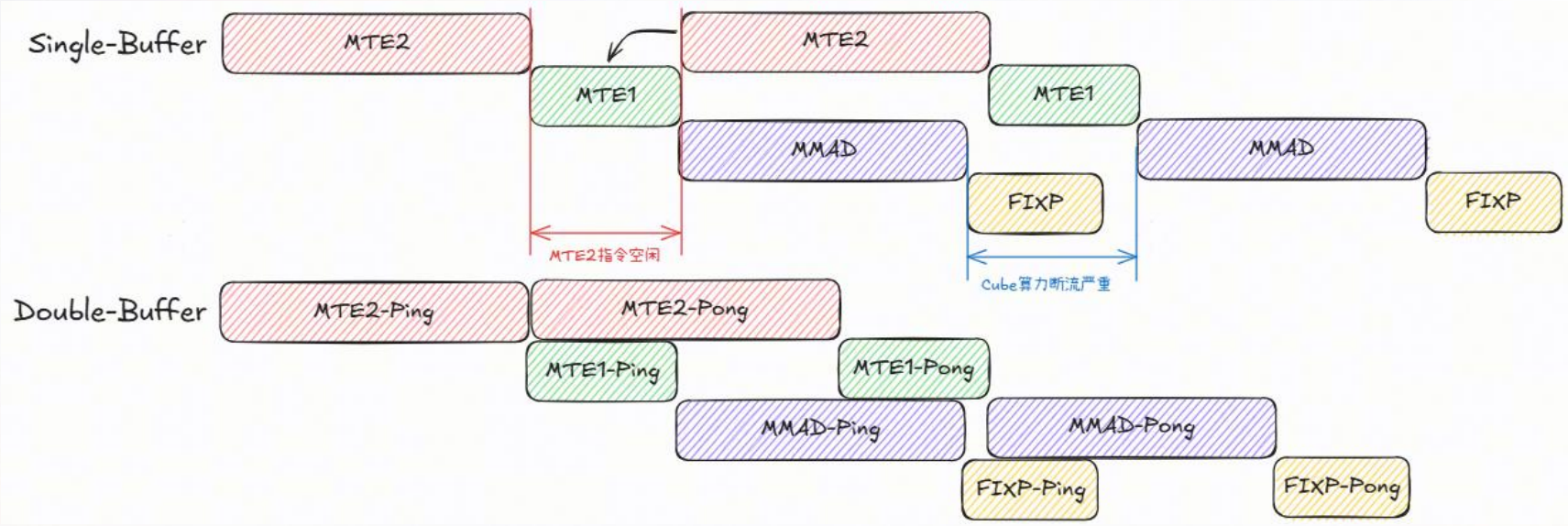
3.4 指令并行度优化：Double Buffer

原理介绍

- Double Buffer使用两个缓冲区交替工作：一个缓冲区用于当前计算，另一个并行准备下一轮数据。
- 通过计算与数据加载/准备的重叠，隐藏内存访问延迟，减少流水线停顿，提高算子吞吐量。

适用场景

1. 存在流水停顿的场景
2. 内存访问延迟成为瓶颈的场景
3. L1或L0缓冲区空间充足的场景



Single Buffer

- 存在MTE2指令不连续的问题
- 存在Cube指令断流问题。

Double Buffer

- 不同Buffer间的数据没有依赖，因此不需要等待。
- 显著缓解了计算断流问题。

干货持续更新，期待在社区与你深度同行

cann-samples

https://gitcode.com/cann/cann-samples

ops-nn

https://gitcode.com/cann/ops-nn

ops-transformer

https://gitcode.com/cann/ops-transformer

Thanks!

访问CANN开源社区

关注昇腾CANN公众号